

研讨：关于整型数

补码的特点

有符号数的编码方法及比较

除了补码 (Two's Complement)，有符号数还有**原码** (Sign-Magnitude) 和**反码** (One's Complement) 两种编码方法。

原码 (Sign-Magnitude)

- 编码规则：** 最高位为符号位 (0 表示正数，1 表示负数)，其余位表示数的绝对值。
- 优点：** 简单直观，容易理解。求绝对值和取反操作非常方便，只需改变符号位。
- 缺点：** 存在**两个零** (+0 和 -0)，例如 8 位原码：0000 0000_2 代表 0， 1000 0000_2 代表 -0。这导致编码空间利用率低。加法和减法运算需要额外判断符号位，使得硬件实现复杂。

反码 (One's Complement)

- 编码规则：** 正数的反码和原码相同。负数的反码是对其绝对值的二进制表示按位取反。
- 优点：** 解决了原码运算中的部分问题，但仍然存在双零问题 (+0 和 -0)。
- 缺点：** 仍然存在**两个零** (0000 0000_2 代表 0， 1111 1111_2 代表 -0)。加法运算需要“循环进位”(end-around carry)，即最高位产生的进位要加回到最低位，这增加了硬件实现的复杂度。

补码 (Two's Complement)

- 编码规则：** 正数的补码和原码相同。负数的补码是对其绝对值的二进制表示按位取反后加 1。
- 优点：** **只有一个零**，即 0000 0000_2，解决了双零问题，编码空间利用率高。将减法运算统一为加法运算。例如， $a - b$ 可以表示为 $a + (-b)$ ，其中 $-b$ 的补码就是 b 的补码取反加 1。这大大简化了计算机的算术逻辑单元 (ALU) 设计，只需要一个加法器就能完成加法和减法。运算规则简单统一，且符合模运算的特性。

特性	原码	反码	补码
0 的表示	两个 (+0, -0)	两个 (+0, -0)	一个
运算规则	复杂，需额外判断	复杂，需循环进位	统一，加法器可完成加减法
硬件实现	复杂	复杂	简单

特性	原码	反码	补码
取值范围	不对称	不对称	对称（负数多一个）

补码的统一加法运算规则和只有一个零的特性使其成为现代计算机表示有符号数的主流方案。

整型数的运算

加法

溢出

假设我们使用 8 位整数，其无符号数表示范围是 0 到 255，有符号数表示范围是 -128 到 127。

无符号数加法溢出

- 设计两个数：200 和 100。
- 计算： $200 + 100 = 300$ 。
- 分析：300 超过了无符号数 8 位的最大值 255。
- 二进制表示：
 - $200 = 1100\ 1000_2$
 - $100 = 0110\ 0100_2$
 - 加法：

```
1100 1000
+ 0110 0100
-----
1 0010 1100
```

- 结论：结果是 9 位，最高位的 1 发生溢出。截断后结果为 $0010\ 1100_2$ ，即 44。结果 300 超出了无符号数表示范围，发生了溢出。

有符号数加法溢出

- 正溢出 (Positive Overflow)：两个正数相加，结果为负数。
 - 设计两个数：100 和 50。
 - 计算： $100 + 50 = 150$ 。
 - 分析：150 超过了有符号数 8 位的最大值 127。
 - 二进制表示：
 - $100 = 0110\ 0100_2$
 - $50 = 0011\ 0010_2$
 - 加法：

```

    0110 0100
+   0011 0010
-----
    1001 0110

```

- **结论：** 结果的最高位为 1，被解释为负数。 $1001\ 0110_2$ 的补码值为 -90 。发生了正溢出。
- **负溢出 (Negative Overflow)：** 两个负数相加，结果为正数。
 - **设计两个数：** -100 和 -50 。
 - **计算：** $-100 + (-50) = -150$ 。
 - **分析：** -150 小于有符号数 8 位的最小值 -128 。
 - **二进制表示：**
 - -100 的补码是 $1001\ 1100_2$ 。
 - -50 的补码是 $1100\ 1110_2$ 。
 - 加法：

```

    1001 1100
+   1100 1110
-----
    1 0110 1010

```

- **结论：** 结果的最高位溢出，截断后结果为 $0110\ 1010_2$ ，这个数是一个正数： 106 。发生了负溢出。

(2) 补码取反操作

$$-x = \sim x + 1$$

要证明 $-x$ 的补码 $B2T(-x)$ 等于 $B2T(x)$ 的按位取反再加 1，即 $\sim B2T(x) + 1$ 。

证明：

数	1	2	3	4	5	6	7	8
x	a	b	c	d	e	f	g	h
$\sim x$	1-a	1-b	1-c	1-d	1-e	1-f	1-g	1-h
$x + \sim x$	1	1	1	1	1	1	1	1
$x + \sim x + 1$	0	0	0	0	0	0	0	0

一个特殊的数Tmin

- $-Tmin = Tmin$
- $Tmin3 = Tmin$ $Tmin5 = Tmin$
- 常用于取反例

1. 左移 (Left Shift)

左移运算符是 $\ll$ 。它将一个数的所有二进制位向左移动指定的位数。

- 规则：
 - 所有位向左移动。
 - 右侧空出的位用 0 填充。
 - 最左侧（最高位）的位被丢弃。
- 效果：
 - 对于无符号数和有符号数来说，左移 n 位都等同于乘以 2^n 。
 - 例如，对于 0000 0101（即 5），左移两位：0000 0101 $\ll$ 2 的结果是 0001 0100，即 20。
 $5 \times 2^2 = 20$ 。
- 溢出：
 - 如果左移操作导致最高位被丢弃，就会发生**溢出**，结果将不再是乘以 2^n 。

2. 右移 (Right Shift)

右移运算符是 $\gg$ 。它将一个数的所有二进制位向右移动指定的位数。

右移分为两种类型，它们的区别在于如何填充左侧空出的位。

逻辑右移 (Logical Right Shift)

- 规则：
 - 所有位向右移动。
 - 左侧空出的位用 0 填充。
 - 最右侧（最低位）的位被丢弃。
- 应用：
 - 通常用于无符号数。
 - 对于无符号数，逻辑右移 n 位等同于无符号除以 2^n 。
 - 例如，对于 8 位无符号数 1100 0000（即 192），右移两位：1100 0000 $\gg$ 2 的结果是 0011 0000，即 48。
 $192/2^2 = 48$ 。

算术右移 (Arithmetic Right Shift)

- 规则：
 - 所有位向右移动。
 - 左侧空出的位用最高位（符号位）的值填充。
 - 最右侧（最低位）的位被丢弃。
- 在 C 和 C++ 中，编译器通常会根据操作数的类型来选择使用哪种右移。对于**无符号数**，它总是执行**逻辑右移**；对于**有符号数**，它总是执行**算术右移**。这确保了右移操作在大多数情况下都能保持数值的正确性。

乘法操作

部分乘法操作作用移位代替

为什么用移位来代替部分乘法操作？

- **移位操作的本质：** 在二进制中，将一个数左移 k 位，相当于将其乘以 2^k 。
- **硬件实现：** 移位操作的硬件实现比乘法简单得多，可以在一个时钟周期内完成，效率远高于复杂的乘法器。
- **优化编译器：** 现代编译器能识别 $x * 2^k$ 或 $x * (2^k \pm 2^j \pm \dots)$ 等表达式，并自动将其优化为更高效的移位和加法/减法指令组合。例如， $x * 15$ 会被优化为 $(x \ll 4) - x$ 。

哪些乘法操作不会被移位代替？

- **乘以非 2 的幂次：** 如果乘数无法被表示为 2 的幂次的和或差，简单的移位操作无法直接代替乘法。
- **浮点数乘法：** 移位操作主要用于整数运算。浮点数的内部表示复杂，无法通过简单的移位来实现乘法。
- **乘数为变量：** 如果乘数是一个变量而不是常量，编译器无法在编译时确定其值，因此必须使用通用的乘法指令。

比较

计算机在进行有符号数和无符号数的比较时，会有一个隐式的类型转换，通常是**将有符号数转换为无符号数**，然后进行比较。

(假设 32 位系统)：

- 有符号数 `signed int a = -10;`

- 无符号数 `unsigned int b = 5;`

1. 转换：

- a 的二进制表示：-10 的补码。32 位下是 `1111...1111 0110_2`。
- b 的二进制表示：`0000...0000 0101_2`。

2. **比较：** 比较时，a 被当作无符号数。将 a 的二进制 `1111...1111 0110_2` 解释为无符号数，这个值非常大，是 $2^{32} - 10$ 。将 b 的二进制解释为无符号数，值为 5。 $2^{32} - 10$ 远大于 5。

3. **结论：** 尽管从数学上看 $-10 < 5$ ，但在 C 语言的比较规则下，表达式 `a < b` **不成立**。

转换

隐式转换

- 有符号数和无符号数进行算术运算时会进行隐式转换

eg:

```
int i
```

```
for(i=12;i-sizeof(int)>=0;i-=4){
```

```
.....
```

```
}
```

扩展和截断

扩展 (Extension)

扩展是将一个 k 位的数转换成一个 w 位的数，其中 $w > k$ 。其原则是保持数值不变。

- **无符号数扩展 (Zero Extension):** 对于无符号数，直接在高位补 0。
- **有符号数扩展 (Sign Extension):** 对于有符号数（补码），在高位补充符号位的值。

截断 (Truncation)

截断是将一个 w 位的数转换成一个 k 位的数，其中 $k < w$ 。其原则是**直接丢弃高位**，即取模运算。

对于一个 w 位的补码数 x ，将其截断为 k 位，结果是 $x \pmod{2^k}$ 。如果截断会改变数值，则会发生溢出。

为什么要用符号扩展？

原因： 符号扩展（相比于直接在高位补 0）是为了**在不同位宽的算术运算中保持数值的正确性**。如果对负数直接补 0，数值就会从负数变为一个很大的正数，导致后续运算出错。

用途：

- **混合类型运算：** 在 C 语言中，当不同位宽的整型数进行算术运算时，会自动进行类型提升。较小的整型数会被扩展到较大的整型数位宽，此时必须使用符号扩展来保证负数的数值不变。
- **乘法和除法运算：** 在进行乘法和除法时，为了防止中间结果溢出，通常会使用比操作数更宽的寄存器。在将 32 位数加载到 64 位寄存器时，必须进行符号扩展以保持其数值。

short int 类型的数的手工转换

short int 类型在大多数系统中是 16 位，即 $w = 16$ 。以下设计几个数，并使用 **B2U** 和 **B2T** 公式进行转换。

正数：2025

- **B2U (Binary to Unsigned):**

由于 $2025 \geq 0$ ，B2U 和 B2T 的结果相同。

$$2025 = 1024 + 960 + 41 = 1024 + 512 \times 1 + 256 \times 1 + 128 \times 1 + 64 \times 1 + 32 \times 0 + 16 \times 0 + 8 \times 1 + 4 \times 0 + 2 \times 0 + 1 \times 1$$

通过短除法：

$$2025/2 = 1012 \dots 1$$

$$1012/2 = 506 \dots 0$$

$$506/2 = 253 \dots 0$$

$$253/2 = 126 \dots 1$$

$$126/2 = 63 \dots 0$$

$$63/2 = 31 \dots 1$$

$$31/2 = 15 \dots 1$$

$$15/2 = 7 \dots 1$$

$$7/2 = 3 \dots 1$$

$$3/2 = 1 \dots 1$$

$$1/2 = 0 \dots 1$$

从下往上读余数，得到二进制表示：0000 0111 1110 1001。

负数：-100

- **B2T (Binary to Two's Complement):**

步骤 1：先求出 $|x| = 100$ 的二进制表示。

$$100 = 64 + 32 + 4 = 0000\ 0000\ 0110\ 0100_2$$

步骤 2：对 $|x|$ 的二进制表示取反（位翻转）。

$$1111\ 1111\ 1001\ 1011$$

步骤 3：在取反结果上加 1。

$$1111\ 1111\ 1001\ 1011_2 + 1 = 1111\ 1111\ 1001\ 1100_2$$

所以，-100 的 16 位补码表示是 1111 1111 1001 1100。

大端模式和小端模式的内存存储

假设我们有一个 4 字节的整型数，值为 $0x12345678$ 。
其四个字节从高位到低位分别是：

- 高字节： $0x12$
- 次高字节： $0x34$
- 次低字节： $0x56$
- 低字节： $0x78$

我们将其存放在内存地址 $0x1000$ 到 $0x1003$ 的位置。

大端模式 (Big-Endian)

- 原则： 高位字节存放在低地址，低位字节存放在高地址。
- 存储方式：
 - 地址 $0x1000$: $0x12$
 - 地址 $0x1001$: $0x34$
 - 地址 $0x1002$: $0x56$
 - 地址 $0x1003$: $0x78$
- 使用场景：
 - 网络通信
 - 嵌入式系统和老式处理器（例如 PowerPC、Motorola 68K 系列）
 - 一些文件格式，比如 Java .class 文件和图像文件（如 GIF、JPEG）

小端模式 (Little-Endian)

- 原则： 低位字节存放在低地址，高位字节存放在高地址。
- 存储方式：
 - 地址 $0x1000$: $0x78$
 - 地址 $0x1001$: $0x56$
 - 地址 $0x1002$: $0x34$
 - 地址 $0x1003$: $0x12$
- 使用场景：
 - 几乎所有基于 Intel x86 和 AMD64 架构（我们的个人电脑）
 - Android 系统：基于 x86 或 ARM 架构的 Android 设备

(9) 跨平台数据传输的格式转换

1. 为什么会进行转换？

- **网络字节序：** 互联网上的数据传输通常使用统一的**网络字节序**，这是一种**大端格式**，确保不同架构的计算机可以正确通信。
- **本地存储：** x86 架构的计算机采用**小端格式**来存储多字节数据。
- **不兼容性：** 如果小端计算机直接读取大端格式的数据，字节顺序会颠倒，导致数据值错误。

2. 在什么地方、什么时候转换？

数据格式的转换通常发生在网络协议栈的**应用层或表示层**，由**软件**来完成。

- **位置：** 由应用程序或网络库（如 BSD Socket API）来完成。C/C++ 开发者会使用 `htonl()` 和 `ntohl()` 等标准库函数进行转换。
- **时机：**
 - **发送数据时：** 在应用程序将数据发送到网络之前，将其从本地字节序（小端）转换为网络字节序（大端）。
 - **接收数据时：** 在应用程序从网络接收到数据后，将其从网络字节序（大端）转换回本地字节序（小端）。

总结： 为了确保跨平台的网络通信正确性，小端架构的计算机在发送和接收网络数据时，必须进行字节序的转换，以适应其本地的存储格式。